

Project Distribution

Shipping your project in the AI coding era

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-05-19

Plan for today

1. **Part 1 — Distributing your project**

- How can a Python project actually leave your laptop?
- A menu of options, with real-world examples

2. **Part 2 — The AI coding era**

- Looking back at what we did, looking forward at where things are going

3. **Second hour** — work on your project, ask questions



No new material today — this is a mainly recap.

Semester Schedule

Lecture	Date	Topic
1: Kickoff	17/2	Course intro, project framing
2: AI assistance and tools	24/2	AI usage models A / B / C
3: Project-based teamwork	3/3	Teams, proposals
4: Intro to Git	10/3	Commits, repos, .gitignore
5: Git continued	17/3	Branches, HEAD, time travel
6: Git collaboration	24/3	Remote, push/pull, PRs
7: AI-assisted Python	31/3	Syntax in the AI era; A/B/C
8: The 3 R's in Python	14/4	Readable · Reusable · Reproducible
9: The Python ecosystem	21/4	pyproject.toml, venvs, imports
10: cont. L9 & feedback	28/4	Packages, project feedback
11: Testing and debugging	5/5	pdb, pytest, tests-as-spec
12: OOP	12/5	Classes you read & use
13: Project distribution	19/5	Distribution + AI era
14: Project marketplace	26/5	Show your work

The final stretch

- **Assignment 5** (OOP rewrite) — peer review due next week
- **Project marketplace** — **26/5** (show your work to everyone)
- **Final project submission** — **29/5** (How? tag your release)

You already have:

- A team, a Git repo, a working project on your laptop
- Code structured as a Python package (L9)
- Tests !! (L11)

What's left: **Distribution**: making your project available.

Part 1 — Distributing your project

A project nobody can run is a project nobody will use.

Today: how can your project be made available to others so they can actually use it?

There are **two principal paths**:

1. **CODE** — user has to install and run it
2. **LINK** — user runs the app directly on their browser

Plus a small “see also” bucket of heavier options.

Ship your Python project

Install the source code

Run on browser

GitHub
repo

pip +
git tag

PyPI

Stream-
lit

Gradio
+ HF

Colab
badge

See also: Zenodo DOI · Docker · PyInstaller

Path 1 — Install the source code

“Here’s my code, run it yourself”

Path 1 — Install the source code

You give the user a URL or a command. They install. They run.

Pros lightweight · no servers to maintain · they get the real source

Cons user needs Python · user must follow your README · install can fail

Three flavours, lowest to highest polish:

1. The GitHub repo itself — `git clone` + manual install
2. `pip install git+...@tag` — pinned to a release
3. PyPI — `pip install yourpkg`

Anatomy of a shippable repo

Path 1 — Install the source code

A “shippable” repo is just a tidy version of what you already have.

```
gempy/
├── gempy/      ← the package code
├── test/       ← tests (yes, singular)
├── docs/       ← documentation
├── examples/   ← example notebooks and scripts
├── requirements/ ← environment / dependency files
├── README.md   ← the front door
├── setup.py    ← install metadata
└── LICENSE     ← legal: "can I use this?"
```

► *Real example: **GemPy git repository** — GemPy is a geoscience library with the same shippable shape (not all shown is here).*

pip install straight from GitHub Path 1 — Install the source code

If your `pyproject.toml` is in order, anyone can install your project with one line:

```
pip install git+https://github.com/gempy-project/gempy.git@v2026.0.1a1
```

What this does:

- Pulls your code from GitHub and installs it like a normal package
- **Pinned to the version tag** `v2026.0.1a1`



*No PyPI account needed. The tag **is** your release moment (L4). For final submission: tag `v1.0`.*

The “real” Python package index. Once you’re there, install is one word:

```
pip install gempy[base]
```

What it takes to publish:

- A PyPI account
- A clean packaging setup (setup.py / setup.cfg)
- `python -m build` to make distributions
- `twine upload dist/*` to push them



*You are **NOT** expected to publish on PyPI.*

Path 2 — Hosting an app on the web

You give the user a URL. They click. The app runs in **their** browser, on **your** hosted server.

Pros zero setup for the user · works on a phone · best demo experience

Cons you have to host it · less reproducible · ill-suited to libraries

Best fit:

- Interactive data exploration (sliders, dropdowns, plots)
- Model demos (input $\rightarrow$ prediction)
- Anything you want a non-coder to try

Pure Python, no HTML or JS. Widgets are one-liners.

```
import streamlit as st

st.title("My project")
x = st.slider("Choose x", 0, 100)
st.write(f"You chose {x}")
```

Streamlit Community Cloud: connect a GitHub repo and deploy.

► *Browse polished examples:* streamlit.io/gallery

 *Additional options: **Gradio + Hugging Face Spaces** for ML-heavy demos, or an **Open in Colab** badge in your README for jupyter notebook projects.*

Heavier paths that exist; we won't dive in today.

- **Zenodo** — archives a tagged GitHub release and assigns a **DOI**. Citable in papers. Very relevant for research projects.
- **Docker** — bundles Python + dependencies + your code into a portable container. Reproducible across machines.
- **PyInstaller** — bundles your project into a standalone `.exe` / `.app`. For end-users with no Python installed.



Example, GemPy ships with a Zenodo DOI badge in its README.

MiniTask: Explore real shipped projects

Open each project. Find one thing:

1. **GemPy** — how does a user install it, and where is the test/folder?
2. **Streamlit gallery** — what does the user see?

Then pair-discuss: which is closest to **yours**?

Part 2 — *AI coding era*

So far today: **how** you ship code in 2026.

Next: **how did you write** that code — and what that means for the way you'll work next.

Three ways we used AI (L7 callback)

From L7 — the three AI usage models:

Model A — Explainer

Ask AI to explain
code.

(“what does this do?”)

Model B — Pair

Inline completion,
prompt-engineered.

(Copilot, Cursor tab)

Model C — Starting point

Generate → dissect
→ understand.

(chat → paste → edit)

Now you’ve used these (or a hybrid) on a real project for $\approx$ 6 weeks.

A fourth model: agentic coding

What's new in 2026: the agent doesn't just complete **a line**. It builds **a feature**.

You describe the goal. The agent reads your repo, edits multiple files, runs tests, iterates.

► *Tools: Claude Code, Cursor agent mode, GitHub Copilot Workspace, Codex, Devin.*

The promise: features in minutes, confident edits across a whole codebase, more output than you alone could produce.

Two arguments worth knowing

Two pieces have been circulating this year. Both written by people who **use AI heavily**. Neither is anti-AI.

1. On “**cognitive debt**” — Margaret-Anne Storey (Feb 2026)
2. “**Agentic Coding is a Trap**” — Lars Faye (May 2026)

Each frames a different cost of going fast with AI assisted coding. We'll take them in turn.

Two kinds of debt



Technical debt — Cunningham (1993)

Lives in the **code**. Messy logic, hurried fixes, shortcuts.

→ the code is hard to **change**.

Cognitive debt — Storey (2026)

Lives in the **team**. Erosion of **shared understanding** of how — and **why** — the system works the way it does.

→ the code is hard to **reason about**.

Story 1 — Cognitive debt in the wild

Part 2 — AI coding era

In Margaret-Anne Storey's entrepreneurship course, a student team hit a wall around **week 7–8**. They blamed **technical debt**. A different problem emerged:

“No one on the team could explain **why** certain design decisions had been made, or how different parts of the system were supposed to work together.”

They had accumulated **cognitive debt** faster than technical debt — and it paralyzed them.

The code **worked**. They just couldn't **change** it any more.

Technical debt

- Lives in the **repo**
- Future-you can **read it**
- Refactor → fix

Cognitive debt

- Lives in your **head**
- When it's gone, it's **gone**
- No refactor — only rebuild

Why AI accelerates it: **you can ship code you don't understand.**

The bug is invisible — until you need to change the code, and the “why” is already lost.



*Where are **you** in your project right now? Could you explain every design choice your team made?*

Lars Faye's argument, in three steps:

1. To use an agent well, you must **supervise it well**.
2. Good supervision needs **strong coding skills** and **critical thinking**.
3. Heavy AI use **atrophies** exactly those skills.

The 'paradox of supervision': effectively using Claude requires supervision — and supervising Claude requires the very coding skills that may atrophy from AI overuse.

— Anthropic study (2025), highlighted by Faye

Reading vs. writing

Old assumption: **writing code is the bottleneck**. AI removes the bottleneck → **productivity** win.

Old assumption: **writing code is the bottleneck**. AI removes the bottleneck → **productivity** win.

Faye's claim:

- Reading **agent** code costs more than reading **your own** code.
- Reviewing **agent** decisions costs more than making them yourself.

Old assumption: **writing code is the bottleneck**. AI removes the bottleneck → **productivity** win.

Faye's claim:

- Reading **agent** code costs more than reading **your own** code.
- Reviewing **agent** decisions costs more than making them yourself.

Why? You wrote your own code — you remember the why. The agent's code arrives without that context; you have to reverse-engineer it.

Faye cites an Anthropic study: a **47% drop-off in debugging skills** when developers *“lean on AI to deliver quick results at the expense of building critical skills.”*

Anthropic surveyed 132 of its own engineers (Aug 2025):

- **59%** of daily work involves Claude (up from **28%** a year before)
- **+50%** productivity gains on average
- ...yet more than half can **fully delegate** only **0–20%** of their work

Having my skills atrophy or fail to develop is primarily problematic with respect to my ability to safely use AI for the tasks I care about.

— Senior engineer, Anthropic (2025)

What this means for you

You already have the tools to **not** fall into either trap. From this course:

L7 — pick your model

A/B/C is still a menu. Agentic is **not** the only option.

L12 — read what you commit

Invest time in understanding the code you generate and commit.

L11 — tests as your contract

Your spec, not the agent's. If AI writes both code **and** tests, what are you testing?

Explain the why

Avoid developing cognitive debt!

MiniTask: Pair up — 5 min

1. Open your project's **git log**.
2. Each of you picks any recent commit (last 2 weeks).
3. Explain to your partner: **why** this change? Not **what** — **why**.

If you struggle, that's not a failure; it's **information**. That's where the cognitive debt is.

What will you do?

AI tools are real. The **productivity** is real. The risks are also real.

The course gave you: git, packaging, tests, reading code, OOP, the AI usage models.

The open questions — for **after** submission, **after** this course:

- When you reach for the agent, will you still be **in the loop**?
- Do you have the **understanding** to review the code you commit?
- Can you still **explain why** you made certain design decision?
- How do you plan to further develop your coding skills?

See you next week for the Marketplace.